

CENTRO UNIVERSITÁRIO UNIFECAP

Cloud DevOps — Orchestrating Containers and Microservices

RENAN GOMES MENDES DE CASTRO

RA: 103686

**Entrega contínua de uma plataforma de pedidos em microsserviços:
do Docker Compose ao Kubernetes com observabilidade e CI/CD**

Parte Teórica — Relatório Técnico

São Paulo

2026

Entrega contínua de uma plataforma de pedidos em microsserviços: do Docker Compose ao Kubernetes com observabilidade e CI/CD

Relatório Técnico — Parte Teórica

1. Introdução e contexto

Este relatório fundamenta a proposta Cloud DevOps para a plataforma “Pedidos Veloz”, da empresa Loja Veloz, um e-commerce que enfrenta indisponibilidades em deploys, dificuldade de escalar em picos de tráfego e baixa rastreabilidade entre serviços. A solução proposta substitui o modelo de implantação manual e heterogêneo por uma cadeia padronizada que vai do ambiente local em Docker Compose até a produção em Kubernetes, com pipeline de CI/CD automatizado, estratégia de deploy de baixo risco, escalabilidade elástica e observabilidade baseada em métricas, logs e traces. As decisões adotadas seguem princípios cloud-native (CNCF) e os doze fatores do 12-Factor App.

2. Microsserviços e o papel do DevOps em ambientes cloud-native

A arquitetura de microsserviços decompõe a aplicação em serviços pequenos, autônomos e alinhados a contextos de negócio (bounded contexts), que se comunicam por APIs leves — síncronas (REST/gRPC) ou assíncronas (mensageria). Cada serviço tem ciclo de desenvolvimento, implantação e dado próprios (database-per-service), o que permite que times trabalhem em paralelo, escolham tecnologias adequadas a cada domínio e implantem mudanças sem coordenar releases monolíticas.

Em ambientes cloud-native — caracterizados por aplicações stateless, elásticas, resilientes e observáveis — o DevOps atua como tecido conectivo entre desenvolvimento e operações. Ele provê cultura (responsabilidade compartilhada), práticas (CI/CD, IaC, postmortems sem culpa) e automação para que o aumento no número de serviços não degrade a confiabilidade. As métricas DORA — lead time, deploy frequency, change failure rate e MTTR — são a régua usada para medir essa maturidade.

3. Containerização: Docker e Kubernetes

Containerização empacota a aplicação e suas dependências em uma unidade portátil e isolada por namespaces e cgroups do Linux. O Docker é a ferramenta de referência para construir e executar imagens OCI em uma única máquina; padroniza o build (Dockerfile), o empacotamento (imagens versionadas em registries) e a execução local. É a base do ciclo de desenvolvimento e do produto final que será implantado.

O Kubernetes, por sua vez, é um orquestrador de contêineres em um cluster de várias máquinas. Não substitui o Docker — opera em uma camada acima: agenda Pods entre nós, mantém o estado declarado (controller pattern), realiza self-healing, atualizações progressivas, escala horizontal e descoberta de serviços. A regra prática é direta: Docker (e Docker Compose) para o ciclo local e para produzir a imagem; Kubernetes para operar essas imagens em produção, com alta disponibilidade e elasticidade.

4. Orquestração de contêineres

O Kubernetes expõe primitivas declarativas — Deployment, Service, Ingress, ConfigMap, Secret, HPA, PodDisruptionBudget — que descrevem o estado desejado. Controllers reconciliam continuamente o estado real com o desejado: se um Pod falha, outro é recriado; se o tráfego cresce, novas réplicas são adicionadas. O Horizontal Pod Autoscaler (HPA) ajusta o número de réplicas com base em métricas (CPU, memória ou métricas customizadas), enquanto o Vertical Pod Autoscaler (VPA) ajusta requests e limits do próprio Pod. ConfigMaps e Secrets externalizam configuração e credenciais, atendendo ao terceiro fator do 12-Factor App.

5. CI/CD em ambientes distribuídos

A Integração Contínua (CI) garante que cada commit seja construído, testado e validado automaticamente; a Entrega/Implantação Contínua (CD) leva o artefato resultante — uma imagem versionada — até o ambiente alvo. Em ambientes distribuídos com muitos microsserviços, cada serviço possui seu próprio pipeline, executado em paralelo, com gates de qualidade (lint, testes unitários, testes de contrato, scan de vulnerabilidades) e uso controlado de secrets.

A estratégia de implantação determina o risco da entrega. Rolling update (padrão do Kubernetes) substitui Pods gradualmente; Blue/Green mantém dois ambientes idênticos e alterna o tráfego instantaneamente; Canary libera a nova versão para uma fração dos usuários e expande progressivamente conforme as métricas confirmam a saúde do release. GitOps (ArgoCD, Flux) leva esse modelo ao limite: o estado do cluster é descrito em um repositório Git, que se torna a única fonte da verdade.

6. Observabilidade: métricas, logs e traces

Monitoramento responde a perguntas conhecidas; observabilidade permite investigar problemas desconhecidos (unknown-unknowns) a partir do comportamento externo do sistema. Seus três pilares são complementares: métricas — séries temporais numéricas agregadas (ex.: latência p95, taxa de erro 5xx), coletadas tipicamente pelo Prometheus; logs — eventos estruturados (JSON) emitidos como streams (12-Factor) e agregados por soluções como Loki ou Elastic; traces — o caminho completo de uma requisição através dos microsserviços, visualizado em Jaeger ou Tempo. O OpenTelemetry (OTel) padroniza a instrumentação dos três pilares por meio de SDKs e do OTel Collector, evitando acoplamento ao vendor de backend.

7. Decisões arquiteturais do projeto Pedidos Veloz

As escolhas a seguir refletem o objetivo de reduzir risco de deploy, encurtar o lead time e aumentar a confiabilidade dentro do prazo de quatro semanas até a campanha promocional:

- **Python + FastAPI:** produtividade alta, OpenAPI nativo, suporte a I/O assíncrono (uvicorn) e instrumentação oficial OpenTelemetry — adequado para os serviços de Gateway, Pedidos, Pagamentos e Estoque.
- **PostgreSQL com schema por serviço:** preserva o princípio database-per-service mantendo um único motor de banco para reduzir custo operacional no MVP.
- **RabbitMQ para o evento PedidoCriado:** desacopla Pagamentos e Estoque do fluxo síncrono de Pedidos; se um consumidor falha, a fila preserva a mensagem e o sistema permanece disponível, aumentando a resiliência.
- **Docker Compose para desenvolvimento local:** sobe toda a stack com um único comando (docker compose up), reduzindo o atrito de onboarding e eliminando o problema de “sobe como dá em máquinas diferentes”.
- **Dockerfiles multi-stage com usuário não-root:** imagens enxutas (python:3.12-slim), camadas reduzidas e superfície de ataque menor; tags semânticas vX.Y.Z + SHA curto garantem rastreabilidade.
- **Kubernetes via Minikube/Kind localmente:** permite validar manifests idênticos aos de produção (EKS/GKE/AKS); readiness/liveness probes evitam encaminhar tráfego a Pods não prontos.
- **HPA por CPU e RPS customizado:** escala automaticamente sob picos da campanha; PodDisruptionBudget protege a disponibilidade durante drains e atualizações de nó.
- **Estratégia de deploy:** rolling update como padrão para serviços stateless e canary via Istio para o serviço de Pagamentos, cuja falha tem maior impacto financeiro — o tráfego é deslocado em incrementos (5% → 25% → 100%) com rollback automático em caso de regressão de SLO.
- **Istio (service mesh) + OpenTelemetry:** mTLS automático entre serviços, traffic shifting para canary, e telemetria coletada do data plane Envoy sem alterar o código dos serviços; o OTel Collector envia métricas para Prometheus, logs para Loki e traces para Jaeger, com Grafana como painel unificado.

- **GitHub Actions:** pipeline com lint (ruff), testes (pytest), build com docker buildx multi-arch, scan de vulnerabilidades (Trivy) e push para GHCR; secrets gerenciados pelo Actions Secrets e injetados no cluster via External Secrets.
- **Terraform (esqueleto):** módulos para VPC, cluster gerenciado e node groups, permitindo recriar a infraestrutura por ambiente (dev/staging/prod) de forma versionada e auditável.

8. Exemplo de referência: Google Online Boutique

A arquitetura proposta inspira-se no repositório público Online Boutique (GoogleCloudPlatform/microservices-demo), uma referência oficial do Google Cloud que implementa um e-commerce de demonstração com dez microsserviços comunicando-se via gRPC, manifests Kubernetes prontos para produção, integração opcional com Istio para mTLS e tracing, e instrumentação OpenTelemetry. O case demonstra na prática os mesmos pilares aplicados ao Pedidos Veloz — contêineres versionados, orquestração declarativa, observabilidade e service mesh — em um contexto de varejo digital comparável ao da Loja Veloz.

9. Conclusão

A combinação Docker Compose (dev) → imagens versionadas (build) → Kubernetes (prod) → CI/CD (GitHub Actions) → observabilidade (OTel/Prometheus/Loki/Jaeger) → service mesh (Istio) materializa, ponta a ponta, os princípios cloud-native e 12-Factor exigidos pelo desafio. A arquitetura proposta reduz o risco do deploy via canary e rolling, viabiliza a escala elástica via HPA, padroniza segurança via Secrets e mTLS, e torna falhas diagnosticáveis via tracing distribuído — atendendo ao objetivo central da Loja Veloz para a campanha promocional.

Referências

- Kubernetes Documentation. Concepts, Workloads, Configuration. <https://kubernetes.io/docs/>
- Docker Documentation. Dockerfile best practices; Compose specification. <https://docs.docker.com/>
- Wiggins, A. The Twelve-Factor App. <https://12factor.net/>
- HashiCorp. Terraform Documentation. <https://developer.hashicorp.com/terraform/docs>
- GitHub. GitHub Actions Documentation. <https://docs.github.com/actions>
- Istio. Traffic Management, Security (mTLS), Observability. <https://istio.io/latest/docs/>
- OpenTelemetry. Specification and Collector. <https://opentelemetry.io/docs/>
- Google Cloud. Online Boutique (microservices-demo). <https://github.com/GoogleCloudPlatform/microservices-demo>
- Forsgren, N.; Humble, J.; Kim, G. Accelerate: The Science of Lean Software and DevOps. IT Revolution Press, 2018.